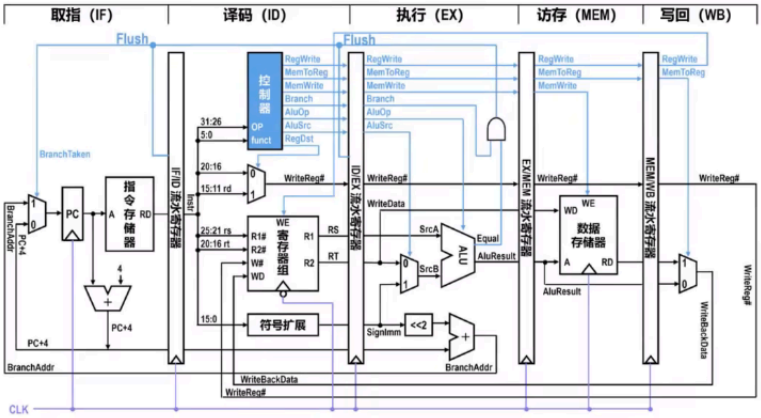
一、产生数据冲突的可能情况

流水线冲突





数据相关类型	涉及操作	产生数据冲突的可能性	在MIPS的标准5段流水线中是否产生数据冲突
RAW	写后读	可能，真依赖	是，并且正常程序中都会存在较多这种数据冲突
WAR	读后写	可能，反依赖（乱序调度流水线）	否，因为按流水线顺序读（ID段）早于写（WB段）
WAW	写后写	可能，输出依赖（乱序调度流水线）	否，因为按序写回（WB段）
RAR	读后读	不可能，结果总是正确	没有讨论的意义

二、如何判断有误数据冲突

判断数据冲突（数据冒险）的本质只有一句话：**后一条指令要用的数据，正是前一条指令要产生、但还没产生的那个结果

第一步：把每条指令的“读集合”和“写集合”标出来

对每条指令标注：

- **源寄存器（Rs, Rt）**：这条指令要**读**哪些寄存器
- **目的寄存器（Rd）**：这条指令要**写**哪个寄存器

例如：

指令	读	写
`ADD R1, R2, R3`	R2, R3	R1
`LW R4, 0(R5)`	R5	R4
`SUB R6, R1, R7`	R1, R7	R6

第二步：逐对检查“后读的 ∩ 前写的”是否非空

对程序中**位置靠前**的指令 i 和**位置靠后**的指令 j，判断：

> **j 的读集合 ∩ i 的写集合 ≠ ∅ ?**

- **是** → 存在 **RAW（读后写）依赖**，即“写-后-读”冲突，这是流水线里唯一真正需要处理的数据冒险；
- **否** → 这两条指令之间没有数据冲突，可以任意流水

注意方向：**只有“前面的写、后面的读”才是冲突。**反过来（前面的读、后面的写，即 WAR）或（前后都写，即 WAW），在**顺序流水线**中不会发生，因为指令严格按序流过各段，读一定发生在写之前、先写的一定先写回

> 补充：WAR / WAW 只有在**乱序执行 / 多发射**（指令可能乱序完成）时才会成为真冒险，此时靠**寄存器重命名**消除。
> 408 里如果题目明确是顺序五段流水线，直接判"无冲突"即可

第三步：有依赖 ≠ 有停顿，还要看"距离"和"类型"

第二步只说明存在依赖关系，是否真的需要**停顿（stall）**，取决于两条指令相隔多远、前一条是什么类型。以**带完整转发（旁路）的五段流水线**为例：

设 j 紧跟在 i 之后，相隔 k 条指令（k = j 与 i 的下标差）：

情况	是否停顿	原因
k = 1，i 是 **ALU / R 型**运算	**不停顿**	i 在 EX 段末就算出结果，可通过 EX/MEM → EX 转发给 j
k = 1，i 是 **LW (load)**	**停顿 1 拍**	数据要到 MEM 段末才从存储器取出，比 j 的 EX 需求晚一拍，转发也救不了（**Load-Use 冒险**）
k = 1，i 是 **Store / Branch**	一般不停顿（分支另算控制冒险）	—
k ≥ 2	**不停顿**	中间隔的指令已经提供了足够的时间，转发路径来得及

所以判断口诀是：

> **先查有无 RAW 依赖 → 再看是否紧邻（k=1）→ 再看前一条是不是 load → 只有"紧邻 + load"才必须停顿 1 拍**

一个完整示例

```
I1: LW   R1, 0(R2)      ; 读 R2，写 R1
I2: ADD  R3, R1, R4     ; 读 R1,R4，写 R3
I3: SUB  R5, R1, R6     ; 读 R1,R6，写 R5
```

- I2 与 I1：I2 读 R1，I1 写 R1 → 有 RAW；k = 1 且 I1 是 load → **停顿 1 拍**；
- I3 与 I1：I3 读 R1，I1 写 R1 → 有 RAW；但 k = 2 → **不停顿**，正常转发；
- I3 与 I2：I3 读 R1，I2 写 R3 → 交集为空 → **无冲突**

容易踩的坑

1. **忘了 load 的特殊性**：很多人以为"有转发就一切搞定"，但 Load-Use 必须停顿，这是选择题高频陷阱
2. **把 WAR/WAW 当冲突**：顺序流水线中判为无冲突；只有题目涉及乱序/超标量时才需考虑。
3. **忽略存储器依赖**：`SW` 后紧跟访问**同一地址**的 `LW`，即使寄存器无交集，也存在存储器数据相关（store-to-load forwarding 或停顿）
4. **把控制冒险混进来**：分支指令导致的是**控制冒险**，不是数据冒险，判断方法不同（看分支目标/条件何时确定、PC 在哪一段更新）
5. **距离算错**：k 是**指令下标之差**，不是时间差；隔了几条独立指令，就多了几拍缓冲

一句话总结

标出每条指令的读/写寄存器 → 检查"后读 ∩ 前写"是否非空（RAW）→ 若相邻且前一条是 load 则停顿 1 拍，其余情况靠转发解决。

三、对于有数据冲突的指令，在什么时机去处理

产生冲突的时间，具体来说，是**后一条指令的 ID 段（读寄存器取操作数）** 与 **前一条指令的结果产生/写回时刻（EX 末或 MEM 末，最终在 WB 段写入寄存器堆）** 撞在了一起

一、为什么是这两个阶段

五段流水线中各段的职责决定了冲突点：

阶段	做什么	与数据相关的动作
IF 取指	取指令	无关
ID 译码	译码 + **读寄存器堆取源操作数**	**消费者在这里“要数据”**
EX 执行	ALU 运算	**生产者（ALU 类指令）在这里“算出数据”**
MEM 访存	读写数据存储器	**生产者（load）在这里“取出数据”**
WB 写回	**把结果写入寄存器堆**	**生产者在这里“正式写入”**

所以 RAW 冲突的本质是：

> **后一条指令在 ID 段就要读到的值，前一条指令要到 WB 段（甚至更晚）才写出来**

即“读”发生在“写”之前，读到的是旧值

二、用时间轴看（紧邻的两条指令）

设 `I1: ADD R1, R2, R3`（写 R1），`I2: SUB R4, R1, R5`（读 R1）：

```
时钟周期:   1    2    3    4    5    6
I1:        IF   ID   EX   MEM  WB
                              ↑ R1 在这里才写入寄存器堆
I2:             IF   ID   EX   MEM  WB
                       ↑ I2 在这里就要读 R1
```

- I2 的 **ID 段在第 3 周期**读 R1；
- I1 的 **WB 段在第 5 周期**才写 R1；
- 第 3 < 第 5 → **读在写之前，冲突发生**

冲突的“位置”就是：I2 的 ID（读）↔ I1 的 WB（写）

三、加了转发后，冲突点“前移”到结果产生时刻

转发（旁路）的意义在于：**不必等到 WB 写回寄存器堆，只要结果算出来就直接送给下一条指令的 EX 入口。**于是判断标准从“WB 何时写”变成“结果何时可用”：

前一条指令类型	结果可用时刻	后一条 EX 需要时刻	结论
ALU / R 型（ADD、SUB…）	**EX 段末**（第 3 周期末）	第 4 周期初	来得及 → **不停顿**
LW（load）	**MEM 段末**（第 4 周期末）	第 4 周期初	**晚一拍 → 必须停顿 1 拍**

load 的冲突时间轴：

```
时钟周期:   1    2    3    4    5    6
I1(LW):     IF   ID   EX   MEM  WB
                              ↑ 数据在第4周期末才从存储器取出
I2:              IF   ID   --   EX   ...
                          ↑ 插入气泡（stall 1 拍）
```

停顿一拍后，I2 的 EX 挪到第 5 周期，正好能用 MEM/WB 转发过来的数据。**这就是 Load-Use 冒险必须停顿 1 拍的根本原因**——不是逻辑上不能转发，而是**数据产生的时刻比需要的时刻晚了一拍**

四、几个容易混淆的点

1. “冲突阶段”要区分“需要的时刻”和“可用的时刻”

- 需求点：后一条指令 **ID→EX 边界**（因为 ALU 在 EX 段就要用操作数）；
- 可用点：前一条指令 **EX 末（ALU 结果）** 或 **MEM 末（load 数据）**，架构上正式写入是 **WB**

2. 没有转发时，所有 RAW 都要停顿 如果题目说明“无转发机构”，那么只要存在 RAW，后一条指令就必须等到前一条 WB 完成后才能读，停顿拍数会明显变多（通常 2~3 拍）

3. ID 段还可能发生结构冒险 若寄存器堆是单端口，某周期同时有“WB 写”和“ID 读”，会产生命名相似的冲突——但那是**结构冒险**（资源不足），不是数据冒险。常见解法是“前半周期写、后半周期读”

4. 分支指令的冲突属于控制冒险 分支条件在 EX 段（或 ID 段，取决于设计）才能确定，此时 PC 更新时机决定了要冲刷/停顿几条指令，判断方法与数据冒险完全不同
